

Control Flow in Python

เจาะลึกการใช้ If-else, For Loop และ While Loop

```
if condition:  
    print("True")  
else:  
    for i in range(5):  
        print(i)
```

คู่มือสำหรับการเขียนโปรแกรมเพื่อการตัดสินใจและการทำซ้ำอย่างมีประสิทธิภาพ



พื้นฐานตรรกะ: Comparison Operators

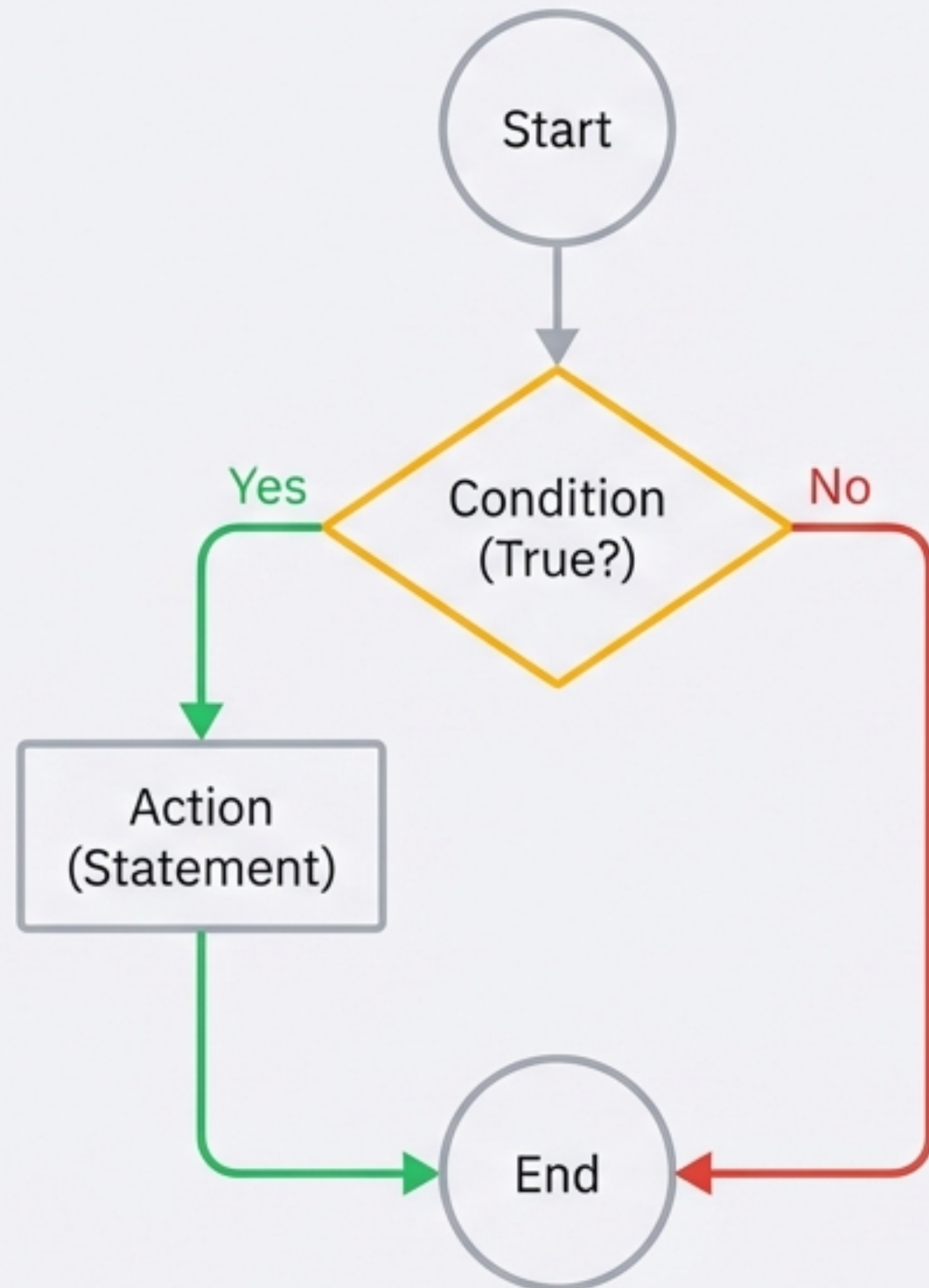
ก่อนที่จะกำหนดเงื่อนไข เราต้อง
ทราบวิธีเปรียบเทียบค่าต่างๆ
ผลลัพธ์ที่ได้จะมีเพียงแค่

True (จริง) หรือ **False** (เท็จ)
เท่านั้น

Operator JetBrains Mono	Description IBM Plex Sans Thai	Thai Meaning IBM Plex Sans Thai
==	Equal	เท่ากับ
!=	Not equal	ไม่เท่ากับ
>	Greater than	มากกว่า
<	Less than	น้อยกว่า
>=	Greater than or equal	มากกว่าหรือเท่ากับ
<=	Less than or equal	น้อยกว่าหรือเท่ากับ

```
print(5 > 3)      # Output: True
print(10 == 0)    # Output: False
```


การตรวจสอบเงื่อนไขด้วย If Statement



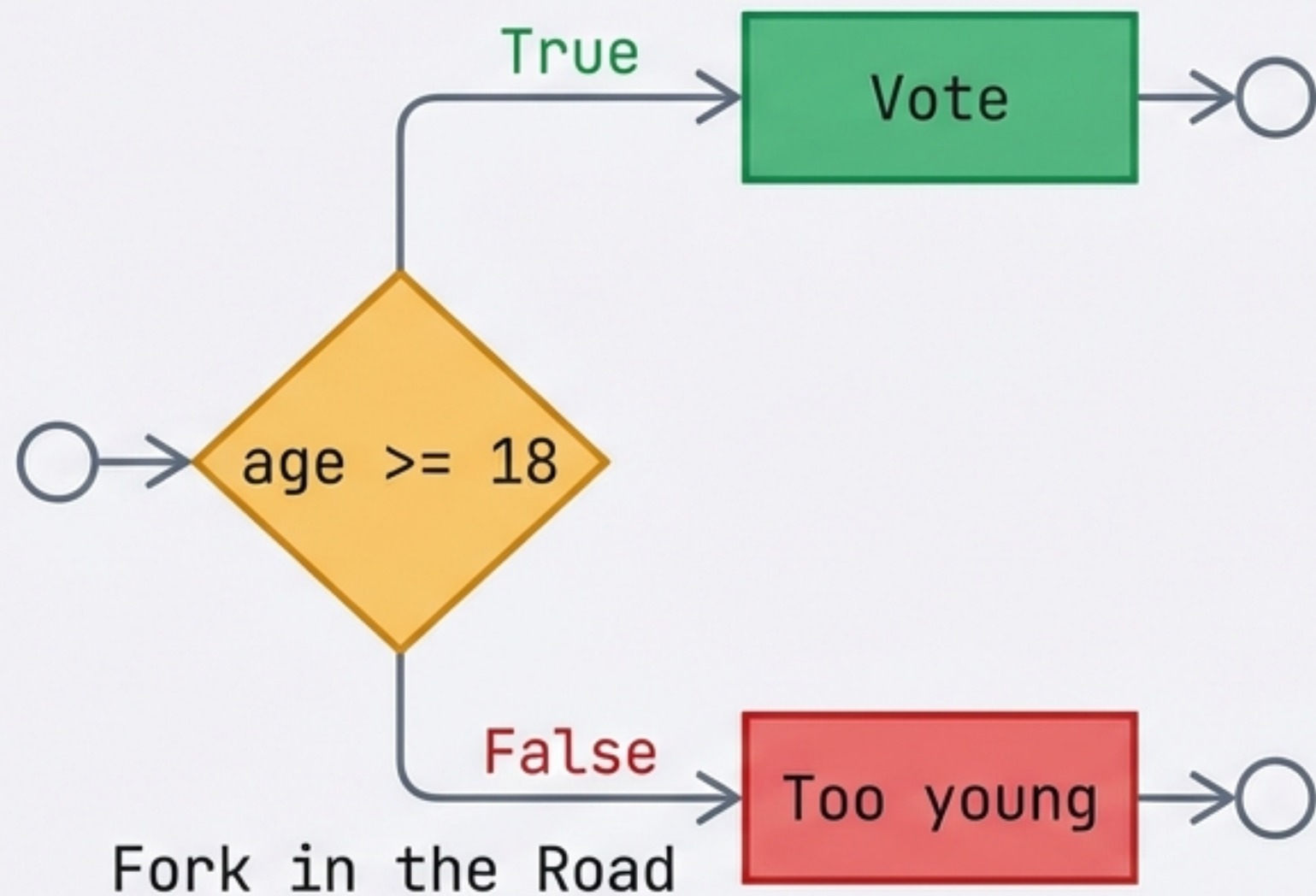
คำสั่ง **if** ใช้กำหนดให้โปรแกรมทำงานเฉพาะเมื่อเงื่อนไขเป็นจริง (True) หากเป็นเท็จ โปรแกรมจะข้ามคำสั่งนั้นไป

```
num = 3

if num > 0:
    print(num, "is a positive number.")

# Output: 3 is a positive number.
```


ทางเลือกสองทาง: If-Else



`else` รองรับกรณีที่เงื่อนไขเป็นเท็จ (False) ทำให้เราสามารถกำหนดการทำงานได้ทั้งสองทางเลือก

```
age = 17

if age >= 18:
    print("You are old enough to vote!")
else:
    print("Sorry, you are too young to vote.")
```


หลายทางเลือก: The If-Elif-Else Chain

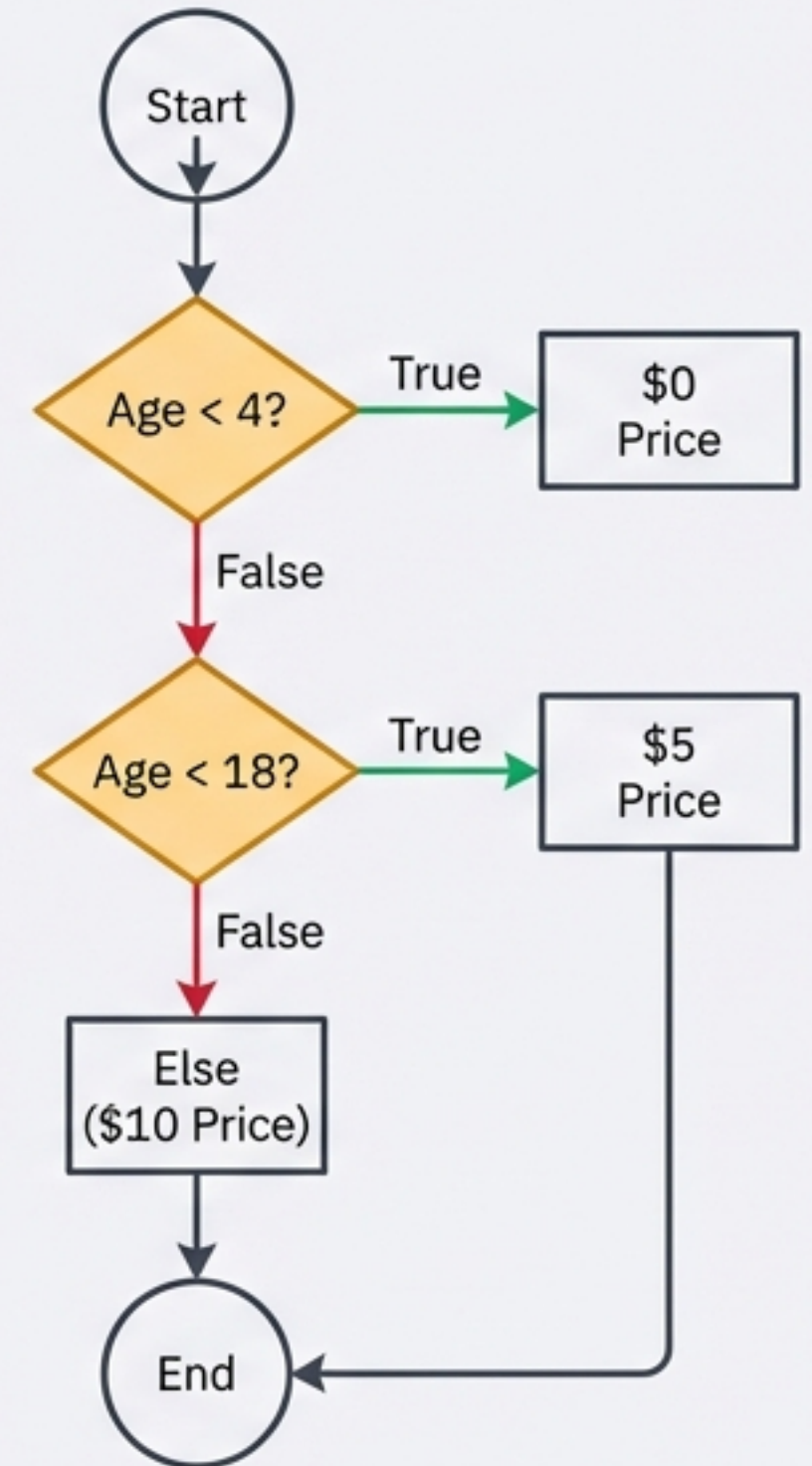
ใช้ `elif` เพื่อตรวจสอบเงื่อนไขเพิ่มเติมตามลำดับ โปรแกรมจะทำงานในบล็อกแรกที่เป็นจริงและข้ามส่วนที่เหลือทันที

Scenario

1. อายุต่ำกว่า 4 ปี: ฟรี
2. อายุ 4 ถึง 18 ปี: \$5
3. อายุมากกว่า 18 ปี: \$10

```
age = 12
if age < 4:
    price = 0
elif age < 18:
    price = 5
else:
    price = 10

print("Your admission
cost is $" +
str(price))
```



Indentation: การย่อหน้าคือหัวใจสำคัญ

Python ใช้การย่อหน้า (Indentation) เพื่อกำหนดขอบเขตของกลุ่มคำสั่ง (Block of code) หากย่อหน้าผิด โปรแกรมจะเกิด `IndentationError` หรือทำงานผิดพลาด



```
if age >= 18:  
    print("Vote") # <--- Aligned perfectly
```

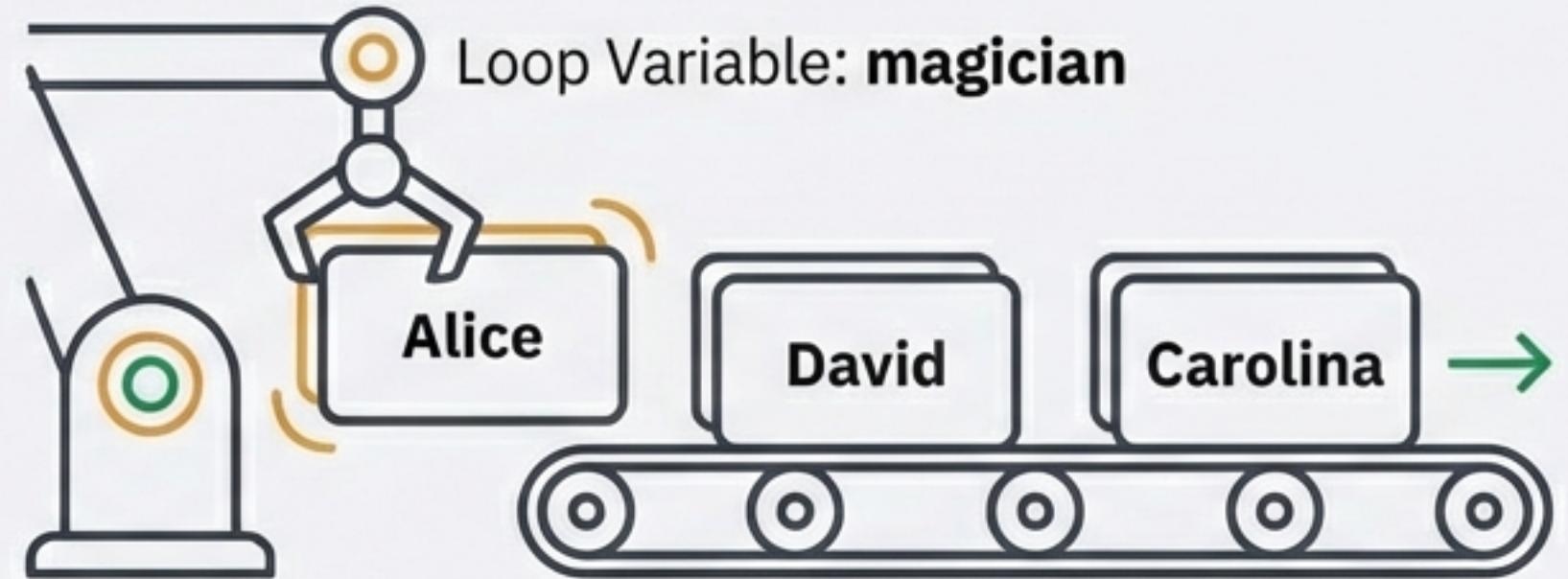


```
if age >= 18:  
    print("Vote")
```

← **IndentationError!**

การทำซ้ำแบบรู้จำนวน: For Loop

ใช้ `for` เพื่อวนซ้ำข้อมูลในลำดับ (Sequence) เช่น List หรือ String ทีละตัวจนครบ



```
magicians = ['alice', 'david', 'carolina']  
for magician in magicians:  
    print(magician.title() + ", that was a great trick!")
```

Output:

Alice, that was a great trick!

David, that was a great trick!

Carolina, that was a great trick!

การวนลูปตัวเลขด้วย Range()

ฟังก์ชัน `range()` ช่วยสร้างลำดับ
ตัวเลขสำหรับวนลูป ระวัง! ตัวเลข
สุดท้ายจะไม่ถูกรวมอยู่ในลำดับ
(Exclusive)



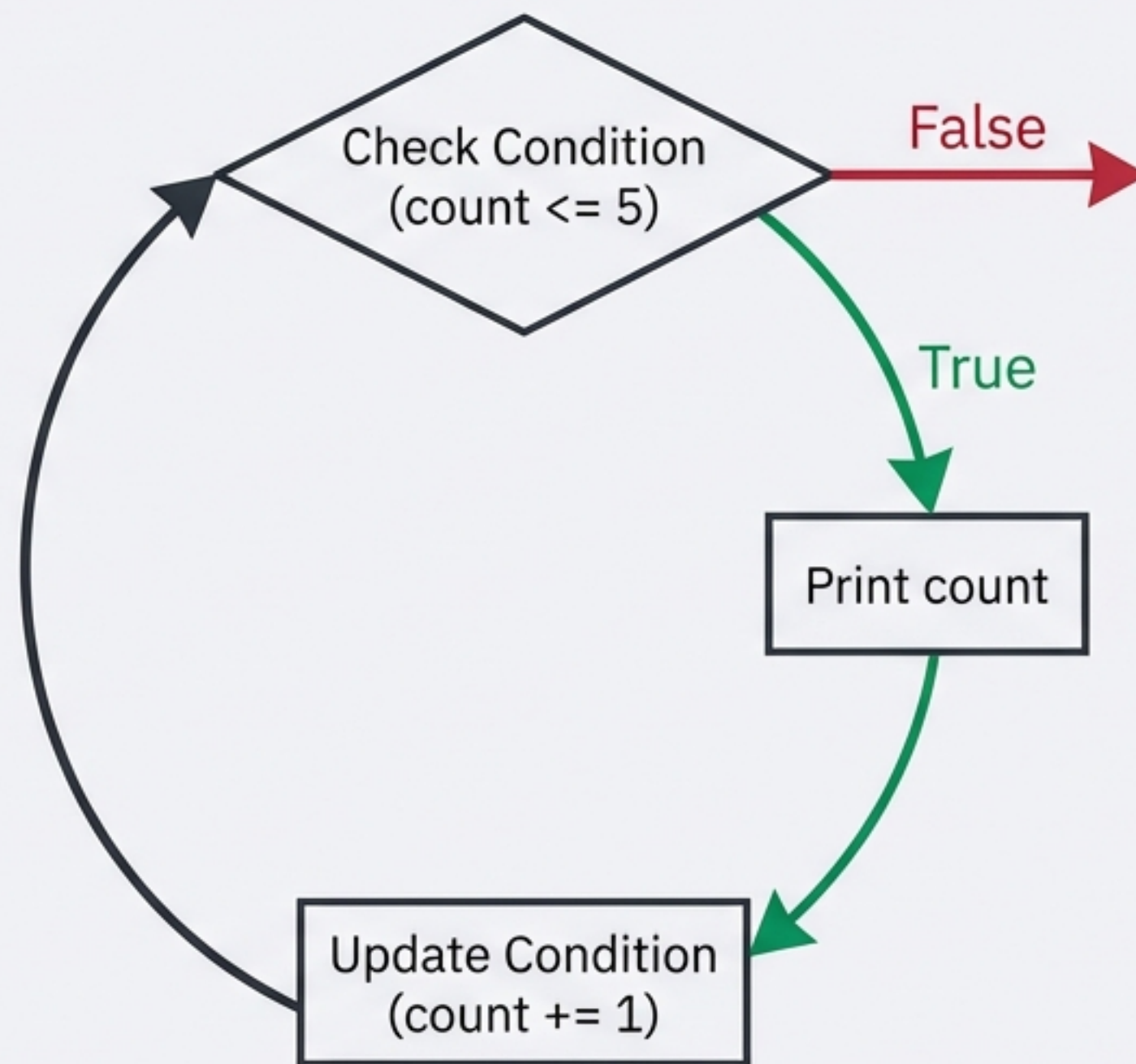
```
for value in range(1, 6):  
    print(value)  
# Output: 1, 2, 3, 4, 5
```

Output: 1, 2, 3, 4, 5

การทำซ้ำตามเงื่อนไข: While Loop

คำสั่ง `while` จะทำงานซ้ำไปเรื่อยๆ ตราบเท่าที่เงื่อนไขยังเป็น **True**

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

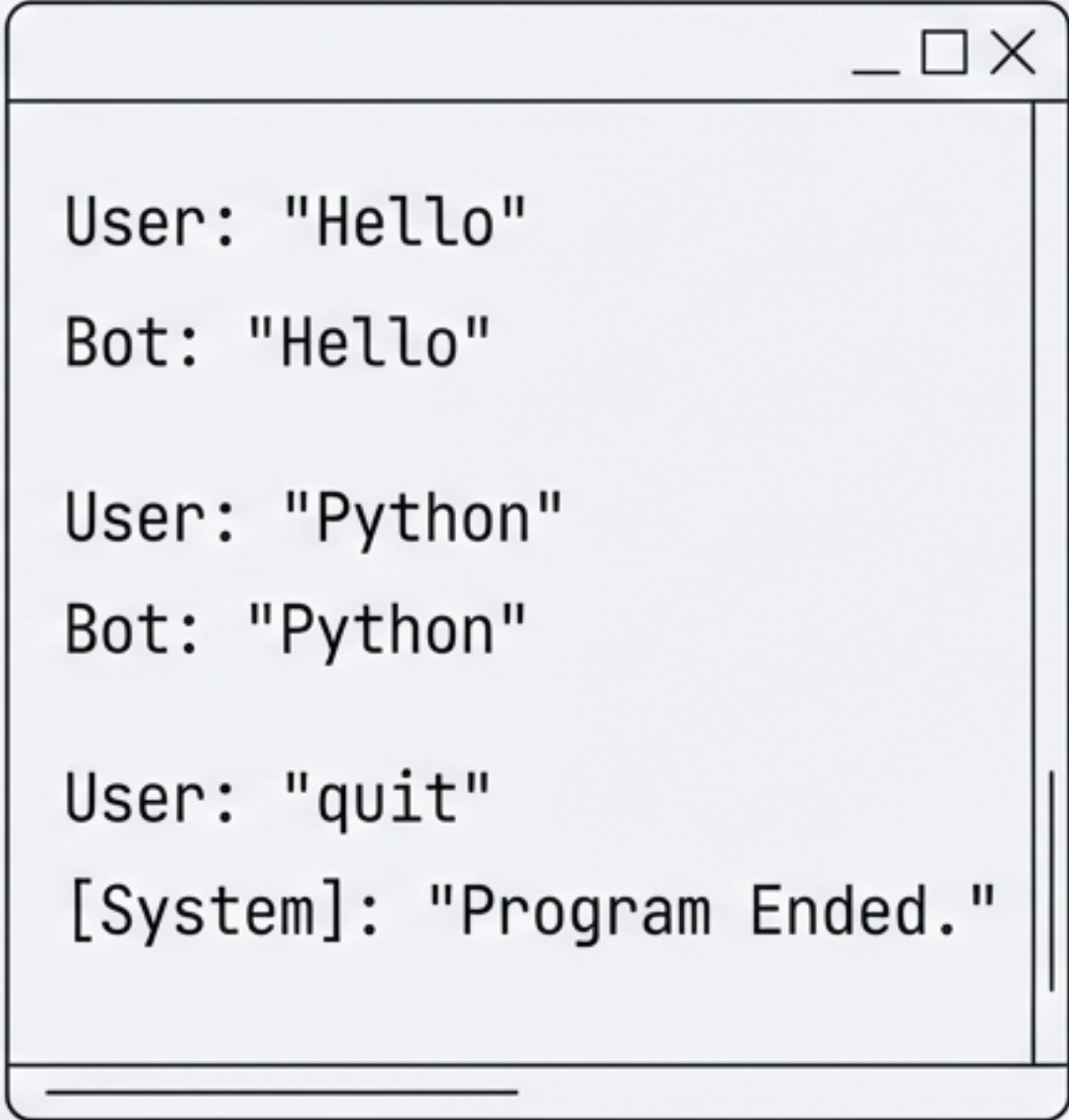


ระวัง Infinite Loop (ลูปไม่รู้จบ) หากลืมเขียนคำสั่งเพื่อเปลี่ยนค่าเงื่อนไข (เช่น `count += 1`)

Interactive Loops: การรับค่าจากผู้ใช้

เรามักใช้ `while` เพื่อรันโปรแกรมต่อเนื่องจนกว่าผู้ใช้จะสั่งให้หยุด (เช่น พิมพ์คำว่า 'quit')

```
message = ""
while message != 'quit':
    message = input("Enter text (or 'quit'): ")
    if message != 'quit':
        print(message)
```



```
User: "Hello"
Bot: "Hello"

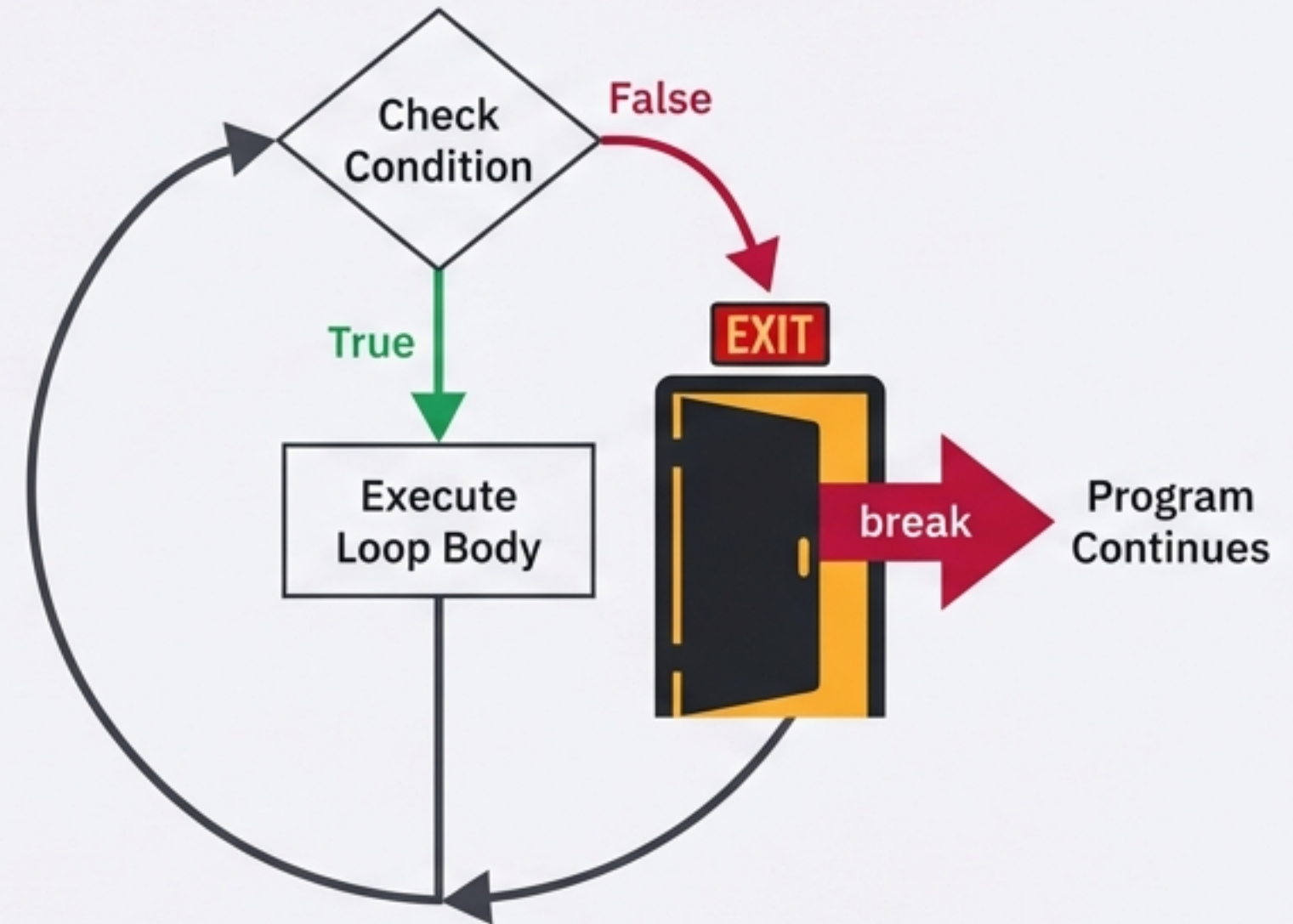
User: "Python"
Bot: "Python"

User: "quit"
[System]: "Program Ended."
```


การหยุดลูบตันที่: คำสั่ง Break

คำสั่ง `break` ใช้สำหรับกระโดดออกจากลูบตันที่ไม่สนว่าเงื่อนไขของลูบจะเป็นจริงหรือเท็จ

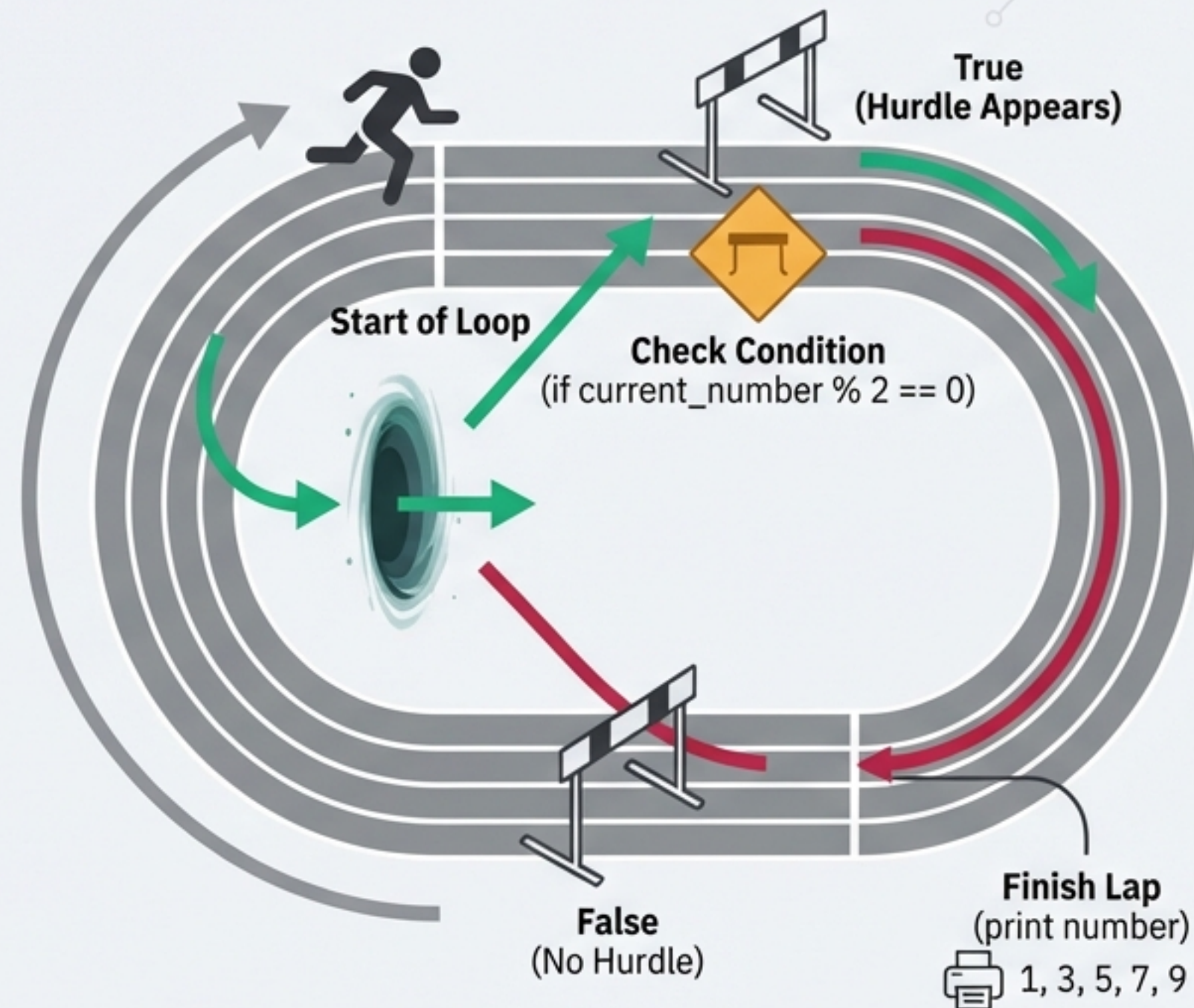
```
prompt = "\nEnter a city you have visited:"  
  
while True:  
    city = input(prompt)  
    if city == 'quit':  
        break  
    print("I'd love to go to " + city + "!")
```



การข้ามรอบการทำงาน: คำสั่ง Continue

คำสั่ง `continue` จะข้ามโค้ดส่วนที่เหลือในรอบปัจจุบัน แล้ววนกลับไปเริ่มต้นรอบถัดไปทันที

```
current_number = 0
while current_number < 10:
    current_number += 1
    if current_number % 2 == 0:
        continue
    print(current_number)
# Prints: 1, 3, 5, 7, 9
```



เปรียบเทียบ: For vs. While

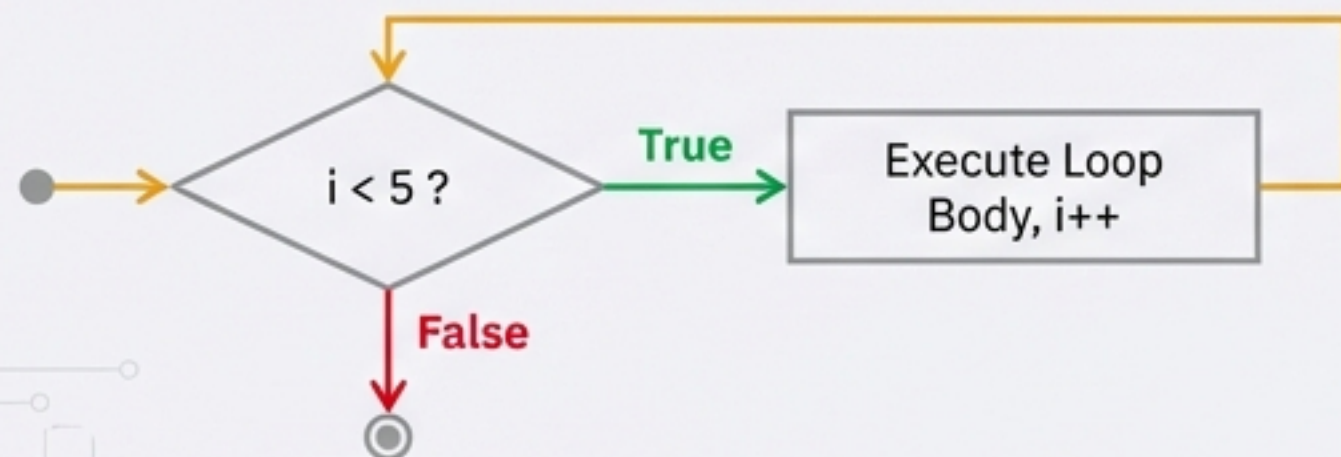
เลือกเครื่องมือให้ถูกกับงาน



For Loop

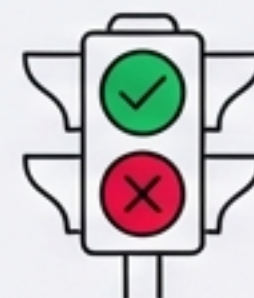
เมื่อรู้จำนวนรอบที่แน่นอน หรือต้องการวนซ้ำข้อมูลใน List (Collection of items)

รู้จำนวนรอบที่แน่นอน



```
for i in range(5):  
    print(i) # 0 to 4
```

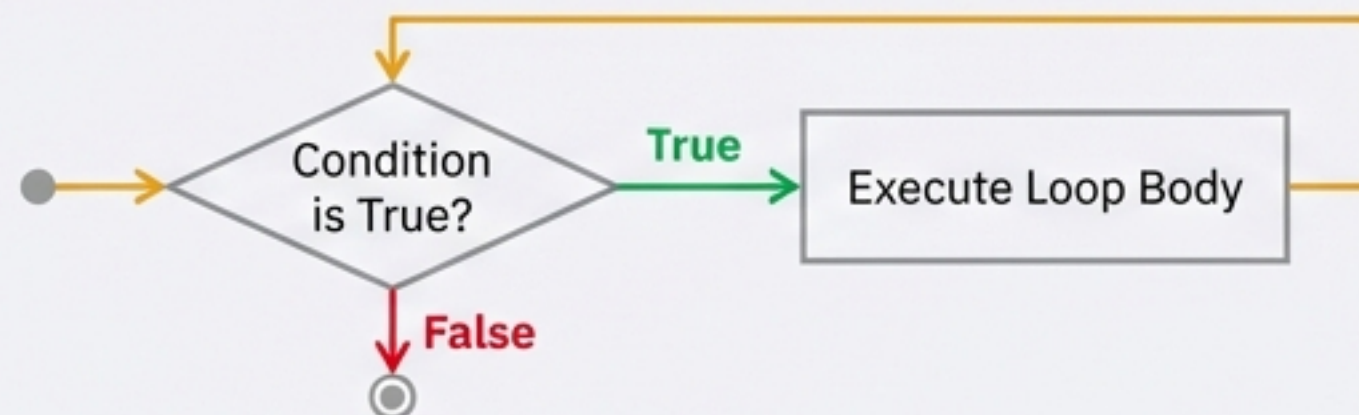
For Loop: ทำงานตามจำนวนรอบที่กำหนด (Known Iterations)



While Loop

เมื่อไม่รู้จำนวนรอบที่แน่นอน หรือต้องรอ User Input / เงื่อนไขบางอย่างให้เปลี่ยนไป

ไม่รู้จำนวนรอบ อาศัยเงื่อนไขหยุด



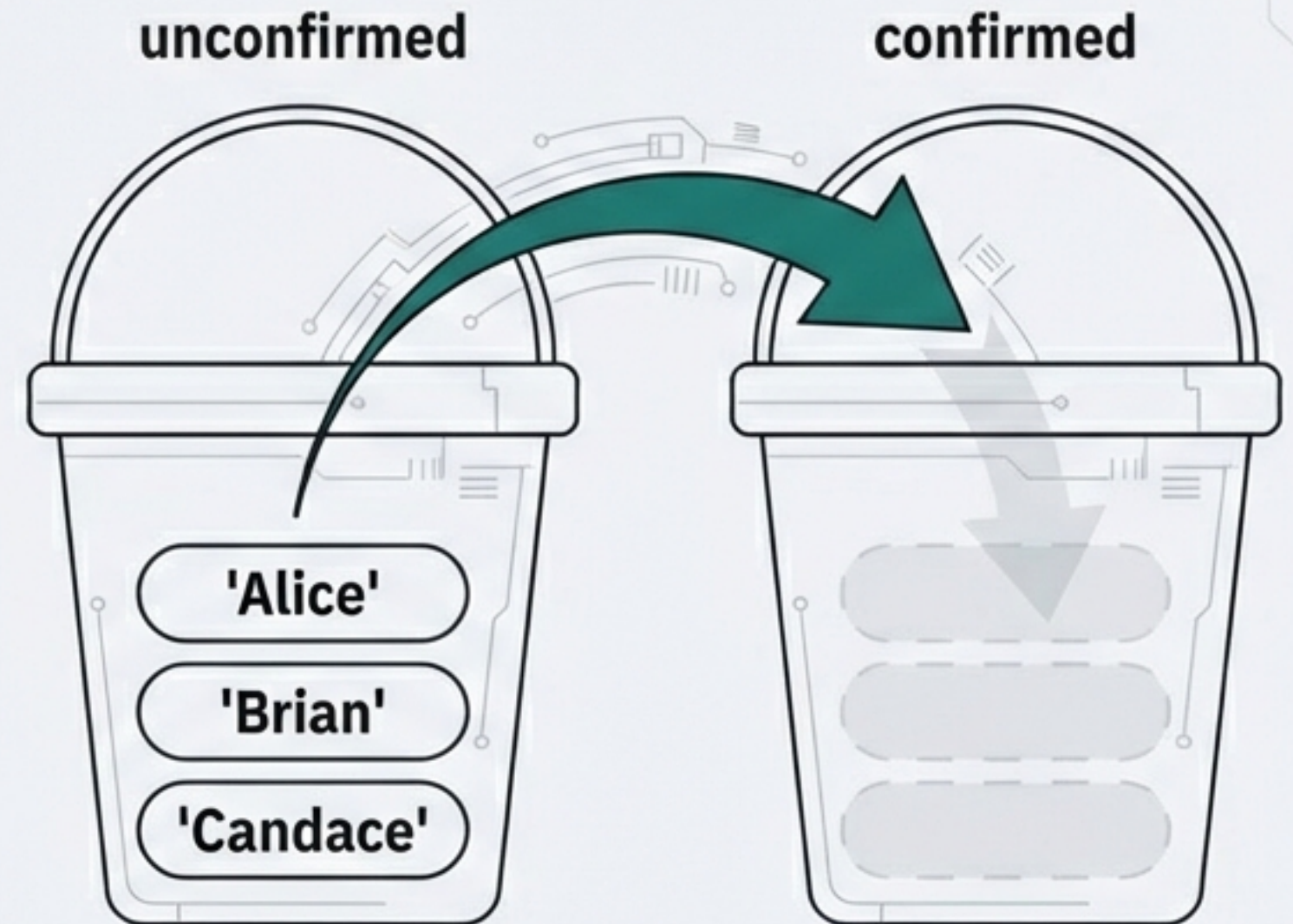
```
active = True  
while active:  
    user_input = input("...")  
    if user_input == 'quit':  
        active = False
```

While Loop: ทำงานจนกว่าเงื่อนไขจะเป็นเท็จ (Condition-Based)

การใช้ Loop จัดการข้อมูลใน List

ไม่ควรแก้ไข List ในขณะที่ใช้ `for` loop วนซ้ำ
หากต้องการย้ายข้อมูลจาก List หนึ่งไปยังอีก
List หนึ่ง ควรใช้ `while` loop

```
unconfirmed = ['alice', 'brian', 'candace']  
confirmed = []  
while unconfirmed:  
    current_user = unconfirmed.pop()  
    confirmed.append(current_user)
```



สรุปไวยากรณ์ (Syntax Cheat Sheet)

Editorial
Technical

Selection

```
if condition:
    # action
elif another_condition:
    # action
else:
    # fallback
```

Logic ตามเงื่อนไขมาทีละที
ถ้าใช่ `another_condition`, ทำ
ในบล็อกขอลากลับ

Repetition

```
for item in list:
    # iterate items
```

```
while condition:
    # repeat while True
```

Loops are iterate items for
condition, `while` condition,
while authors repeat while True

Control

- **break**
Exit loop (ออกจากลูป)
- **continue**
Skip iteration (ข้ามรอบนี้)
- **pass**
Do nothing (ข้ามไปเลยๆ)